CP2 Development Information: TimeTravel

Repository

- **GitHub repository**: https://github.com/abhiverma13/TimeTravel
- **Commit SHA for this submission (main)**: 86b568c57cb1fac7c5393c09b821071e55f5483e

Back-end server

- **Public IP**: 136.67.87.217 (Google Cloud VM, static IP)
- **Domain name**: none
- **Health check**: https://136.67.87.217/health returns `{"status":"ok","database":"connected"}`. The server uses a self-signed certificate (see "Configuration" below), so a browser shows a certificate warning.

How the project scope requirements are implemented

Requirement	Use cases	How
External authentication	Sign In	The app signs in with Google through Credential Manager. The back-end verifies the Google ID token with google-auth-library (`POST /api/auth/google`) and issues its own 7-day session token, which every other API requires.
External API	Browse & Search Activities, Generate Itinerary, Monitor Trip Conditions	**Implemented.** The Activity & Review Service builds each city's catalogue from Google Places API (New) Text Search, one search per interest tag, cached for 7 days; the Itinerary Engine looks up the lodging with Places. Travel times come from the Google Routes API (Compute Route Matrix), cached for 7 days, with an offline estimate as fallback. The disruption timer reads the OpenWeather 5-day forecast and each venue's business status from Places.
Changes in response to external events	View & Adjust Itinerary, Notify Trip Party, Re-plan Itinerary	**Implemented.** A member's adjustment is saved with a version check, and the Trip Sync & Disruption Service pushes the new plan over Socket.io to every member's open app within 2 seconds. Every 30 minutes, a timer checks the forecast (OpenWeather) and venue status (Places) of stops on trips under way; a disrupted stop produces a proposed day, pushed live, that members accept or dismiss.
Own computation	Generate Itinerary (Rank & Schedule Day Plan), View & Adjust Itinerary, Add Place to Itinerary, Re-plan Itinerary	**Implemented.** The Itinerary Engine (`backend/src/itineraries/engine/`) solves an Orienteering Problem with Time Windows: it prunes candidates (within 25 km, open during the trip, top 40 by community score times interest match), builds each day by greedy insertion by score density from three starting rules, and improves it with reorder and swap local search under a 200 ms per-day budget. Adjustments, added places and disruption proposals repair only the affected day, keeping locked and started stops. Group trips weight each interest by the share of members who chose it and leave out every member's vetoes. Community scores are a Bayesian average that shrinks sparse ratings toward the city mean.

Configuration for building and deploying

The README's "Cloud deployment (HTTPS)" section has the full steps for a fresh VM.

Back-end (`backend/.env`, from `backend/.env.example`):

Variable	Value
`PORT`	443 (3000 for local HTTP development)
`GOOGLE_CLIENT_ID`	986175081923-12poichbpt3ej109488ist07e5omivqh.apps.googleusercontent.com (the OAuth web client)
`JWT_SECRET`	Any long random string, for example the output of `openssl rand -hex 32`. Each deployment generates its own.
`GOOGLE_PLACES_API_KEY`	Optional. A key with the Places API (New) and the Routes API enabled. Our key is restricted to our VM's IP, so it does not work from another machine. Without a key, run `docker compose exec backend npm run seed:demo` to load the Vancouver demo catalogue; travel times are then estimated. We can provide a key for testing on request.
`OPENWEATHER_API_KEY`	Optional. The free plan's 5-day forecast is enough. Without it, trips under way are checked for closed venues only, not for rain. We can provide a key for testing on request.
`TLS_CERT_PATH`	`/certs/server.crt` (mounted from `backend/certs/`)
`TLS_KEY_PATH`	`/certs/server.key`
`MONGODB_URI`	Leave as in the example. Docker Compose provides MongoDB 7.0.

- **Certificate**: create a self-signed certificate for the VM's public IP in `backend/certs/`. The `openssl` command is in the README.
- **Start**: run `./scripts/run-backend.sh`.
- **Optional demo data**: `docker compose exec backend npm run seed:demo`.

Front-end (`frontend/local.properties`, from `frontend/local.properties.example`):

Property	Value
`API_BASE_URL`	https://136.67.87.217
`GOOGLE_CLIENT_ID`	986175081923-12poichbpt3ej109488ist07e5omivqh.apps.googleusercontent.com
`MAPS_API_KEY`	Optional. A Maps SDK for Android key restricted to package `com.example.cpen321application` and the SHA-1 of the signing key. Without it, the trip map shows no map tiles. We can provide ours on request.
`RELEASE_*`	Only needed to build a signed release APK.

- **Trusted certificate**: the app trusts our server's self-signed certificate through the copy bundled at `frontend/app/src/main/res/raw/backend_certificate.pem`. A build pointed at a different server needs that file replaced with the new server's certificate.
- **Google sign-in, maps and signing keys**: sign-in and map tiles only work for builds signed with a key whose SHA-1 is registered on our Android OAuth client and our Maps key. The submitted

`CP2_TimeTravel.apk` is signed with our registered release key. A TA who builds their own debug APK can send us their debug SHA-1 (from `./gradlew signingReport` in `frontend/`) and we will add it.

Accounts

- TAs sign in with their own Google accounts.
- The OAuth consent screen is in Testing mode, so only listed test users can sign in. Test users added for the TAs: We will add each TA's Google account as a test user as soon as they send it.
- Groups, friends, the feed and Nearby involve two travellers, so they need two signed-in accounts (for example two TAs, or one account on the emulator and one on a phone).
- The trip map, Nearby and verified reviews use the device's location. On the emulator, set it in Extended controls > Location (for example to Vancouver).
- No other accounts are needed.

Physical device

- Google Pixel 9

Humblebrag

- **The itinerary engine finds the best plan.** On 25 seeded random single-day instances, it matches the exact optimum found by brute force. A 5-day, 40-candidate plan is generated in about 0.4 s at the 95th percentile, measured at the API.
- **Live edits never overwrite each other.** Every itinerary has a version number, and an edit from an out-of-date copy is refused and reloaded. Changes go through the REST API, so each one is validated in one place, and Socket.io only broadcasts the result: the NFR 2 test delivers 60 updates with a 95th percentile of 77 ms.
- **Disruptions are suggestions, not surprises.** A proposed day is saved with the trip version it was made from, so it lapses if anyone changes the trip first, and after 12 hours. A wrong forecast cannot rearrange a day by itself.
- **Verified visits are hard to fake.** A review is verified only if it is written within 150 m of the place on a day the place is in the reviewer's trip, so passing by is not enough.
- **Privacy by design.** Shared positions are relayed live and never stored, and nearby travellers see a trip's plan but never its lodging or anyone's position.
- **It degrades instead of failing.** A stale catalogue is served when Google Places is down, travel times fall back to an offline estimate when Google Routes fails, and a lodging that cannot be found falls back to the city centre with a warning.

Limitations

- **Travel times**: Google Routes times cover the lodging and the 12 best candidates; other pairs use the offline estimate. Adjustments use cached times only.
- **No push notifications**: live changes and suggestions reach only an open app; others see them the next time they open the trip.

- **Location sharing** works only while the trip map is open; nothing is tracked in the background.
- **Disruption checks** run every 30 minutes for stops in the next 36 hours, using OpenWeather's 3-hour forecast steps.
- **Groups and Nearby**: a group holds at most 12 members, and Nearby shows only trips with a stop within 10 km that start within 7 days.
- **Without a Places API key**, only cities already in the database (for example the Vancouver demo catalogue) can be browsed or planned.
- **Trip length and time zones**:
  - Trips are limited to 7 days.
  - Each trip is planned in the destination's local time, without time-zone conversion.
  - Opening hours that run past midnight are cut at midnight.
- **Reviews**: one review per traveller per activity; reviewing again edits it.
- **Sign-in** is limited to listed test users while the OAuth consent screen is in Testing mode.
- **Self-signed certificate**: the app trusts only the bundled certificate, so moving the back-end to a new server requires rebuilding the app.
- **Container user**: the back-end container still runs as root (see Code_Review.md 2.4); running it as the `node` user is planned for the next deployment.